The Reported–Deployed Gap in Phishing URL Detection: A Time-, Source-, Provenance- and Evasion-Aware Benchmark with Drift-Aware Selective Hardening

Rajarajeswari V
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India
rajarajeswarivisu@gmail.com

Jeff Adrain G
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India

Arivu Nithi M
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India

Kevin Abishek K
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India

Ms. Meena Priyadharsini
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India

Dr. T. Sudalaimuthu
Dept. of Computer Science and Engineering
Hindustan Institute of Technology and Science
Chennai, India

Abstract—Published phishing URL detectors routinely report 97–99.5% accuracy, yet these figures are almost always obtained under a single evaluation protocol: a random or k-fold split of one dataset, balanced at roughly 50% phishing prevalence. We audit a corpus of ten recent papers — eight method or empirical studies published between 2024 and 2026, plus two reviews — and find that none evaluates under a temporal split, none reports precision at a realistic deployment base rate, none tests its own detector against the evasion techniques the literature catalogues, and only one performs genuine cross-source transfer — where reported accuracy falls by roughly twenty percentage points. We introduce PhishDriftBench, a four-axis protocol that evaluates detectors across time, across data sources, against both rule-based and generative evasion, and at deployment-realistic class priors. Under this protocol we re-evaluate five representative architectures spanning gradient boosting, layered brand-jacking screens, residual MLPs, cross-dataset feature intersection and transformer–GBM hybrids. We further contribute a feature-provenance audit that separates statically computable lexical features from those requiring a live lookup, and quantifies how much published accuracy is attributable to lookup artifacts — notably retrospective WHOIS and DNS resolution, which correlates with domain mortality rather than phishing semantics. Finally we propose DASH (Drift-Aware Selective Hardening), a lightweight mitigation combining drift-stability feature weighting, unsupervised drift detection, a small active-labelling budget and a calibrated abstention band. Our temporal (Axis T) run was inconclusive by construction — every baseline sat within a 0.003 AUC band of ceiling at every horizon against a task too separable to leave temporal drift room to appear — but cross-source transfer (Axis S) was not: AUC fell by up to 8.9 points depending on model and direction, one baseline's transfer cost inverted with direction rather than scaling with it, and only a BERT-based hybrid stayed within 1.1 points of in-distribution performance in both directions. At a deployment-realistic prevalence of one phishing URL in ten thousand, precision for three of five baselines falls to 4.8–11.2%, meaning roughly nine in ten alerts would be false, despite 98–99% training-time accuracy that is statistically indistinguishable across all five models — accuracy is shown to be actively uninformative about deployment false-alarm rate on this corpus. Rule-based evasion (Axis E1) mostly failed to reduce recall for any baseline, and two transforms increased it, because they added exactly the signal these feature sets already treat as suspicious; the one baseline with a real single-transform vulnerability (homoglyph substitution, -2.75 points) is also the only baseline whose cross-source transfer cost inverts with direction. By contrast, a simple generator trained on recent real phishing text (Axis E2) costs every baseline 4.5–5.4 recall points relative to one trained on older phishing text from the same corpus — the one axis where all four baselines tested degrade uniformly, including the otherwise-robust BERT-based hybrid. DASH could not be credited with recovering degradation on the one temporal stream tested, because that stream showed none to recover — itself a consequence of the same separability that makes the temporal result inconclusive, not a failure of the mitigation. Because none of the three acquired corpora carry genuine lookup-based fields, we performed 285 live WHOIS/DNS/TLS lookups ourselves: lookup-based features did not improve accuracy over lexical features alone (AUC -0.0051), and lookup success flags alone reached only 0.56 AUC — a negative result for the domain-mortality hypothesis on this sample — though a class-conditional breakdown surfaced a real, survivorship-induced reversal in DNS/SSL success between old and recent phishing submissions that traces to how the source corpus

itself was filtered, not to domain age directly. Finally, we build B6, a model that adds exactly one fix per measured weakness and evaluates each addition in isolation: cleaning the training data alone does not improve precision; stability-weighted training has little effect once a model is not itself confounded by cross-source training as B3 was; adversarial generative augmentation nearly halves the Axis E2 recall gap (a -0.049 drop becomes -0.026) but increases the false-positive rate; and a two-stage cascade — discarding an initial heuristic second stage that collapsed recall to 0.757 in favour of a properly trained classifier — recovers that cost and cuts false positives sixfold, raising precision at $\pi = 10^{-4}$ from 1.8% to 9.95%, surpassing three of the five original baselines. A further proposal, B7, adds an unsupervised novelty gate targeting zero-day phishing specifically; it recovers 5.6 recall points against a synthetic zero-day proxy but, fed through the same prevalence-correction math, collapses precision to 0.20% — a $\sim$50$\times$ regression — while a refinement narrow enough to avoid that cost also erases the benefit. We report this as a decisive negative result, not a tuning failure: on this feature set, "looks unlike normal legitimate traffic" is not separable enough from ordinary legitimate diversity to support a low-cost gate.

Index Terms—phishing detection, URL classification, concept drift, dataset shift, evaluation methodology, adversarial robustness, machine learning security, data leakage, benchmark

I. Introduction

Phishing remains among the most economically damaging classes of cybercrime, and URL-based detection is attractive because it operates before any content is fetched or rendered, imposing negligible latency and no rendering infrastructure. A large body of work reports that this problem is close to solved: accuracies of 98.29% [1], 99.45% [2], 99.48% [3] and 99.35% [4] have all appeared in the last two years.

These numbers are not fraudulent, but they are answers to a question that is not the deployment question. Almost without exception they are obtained by randomly partitioning a single corpus, collected over a single time window, balanced at roughly equal class prevalence, and evaluated with no adversary in the loop. Each of those four choices is individually defensible and collectively misleading:

1) Random splits leak time. Phishing URLs arrive in campaigns: one kit, one registrar burst, one TLD fashion, thousands of near-identical strings. A random split places members of the same campaign in both training and test partitions, so the model is graded on near-duplicates of what it memorised.
2) Single-source evaluation hides source-specific artifacts. A feature can carry opposite meaning in different corpora. Misiek and Hyla [5] report that HTTPS presence characterises 100% of legitimate URLs in PhiUSIIL but only 59.2% of legitimate URLs in their own corpus.
3) Balanced test sets hide the base-rate problem. A detector with 99% accuracy and a 1% false-positive rate, deployed at a phishing prevalence of $1{:}10^4$, yields a precision near 1% — roughly ninety-nine false alarms for every true detection. Reported accuracy carries no information about this.
4) Adversaries are catalogued but never applied. Li et al. [6] systematically review cloaking and evasion techniques; no method paper in our corpus evaluates its own detector against them.

Exactly one paper in our corpus measures the consequence. Misiek and Hyla [5] report 97.80% under single-dataset evaluation and 77.84% averaged across three sources — a 19.96 percentage-point collapse that they correctly attribute to dataset shift, "a factor often overlooked in existing literature."

A. Contributions

- C1. An evaluation-protocol audit of ten recent phishing detection papers, eight of them method studies from 2024–2026 (Table I), establishing that the protocol gap is current rather than historical.
- C2. PhishDriftBench: a four-axis protocol evaluating detectors across time (Axis T), across sources (Axis S), against rule-based and generative evasion (Axis E), and at deployment-realistic class priors.
- C3. A feature-provenance and lookup-dependency audit that partitions features by whether they require a network lookup, and quantifies the accuracy attributable to retrospective-lookup artifacts.
- C4. A drift-stability scoring method that identifies drift-brittle features before deployment.
- C5. DASH, a lightweight drift-aware mitigation combining stability-weighted features, unsupervised drift detection, a bounded active-labelling budget and a calibrated abstention band.
- C6. B6, a proposed model built incrementally so that each component's individual contribution is measured rather than assumed: data cleaning, stability-weighted training, adversarial generative augmentation, and a two-stage cascade whose second stage is consulted only for the subset Stage 1 does not confidently clear.
- C7. B7, an unsupervised novelty gate proposed for zero-day phishing and evaluated against the same deployment metric the rest of this paper uses, showing that its recall benefit does not survive contact with prevalence-corrected precision.

II. Related Work and Protocol Audit

A. Feature-engineering and classical ML approaches

Kustiawan and Ghauth compare URL-, HTML- and derived-feature sets across ten classifiers [2] and subsequently propose PhishOFE [3], an optimised feature-engineering framework that deliberately avoids third-party features, reaching 99.48% with CatBoost. Goenka et al. [4] propose a two-layer model in which a domain-squatting and URL-obfuscation screen precedes a lexical classifier, reporting 99.35% at 12.49 ms. Notably, their model attains 99.35% on their own 2025 corpus but 89.17% on an older

corpus, which they attribute — correctly — to the top targeted brands of 2025 differing from those of 2018. This is a direct observation of concept drift, measured but untreated.

B. Deep and hybrid architectures

Remya et al. [1] apply a residual pipeline of convolutional and inverted-residual blocks feeding an MLP over lexical, sentiment and domain-age features. The same group later proposes BGL-PhishNet [7], fusing BERT, a graph neural network and LightGBM, reporting 97.3% on a 549,346-URL corpus. He et al. [8] introduce a TextRank-based feature-extension library feeding a BERT–CNN classifier. Vulfin et al. [9] fuse four modalities — CatBoost over URL features, a 1D CNN over characters, CodeBERT over HTML and EfficientNet-B7 over screenshots — with SHAP and Grad-CAM explanation, and are among the few to quantify temporal decay, reporting an 8–15% degradation.

C. AI-generated phishing URLs

Early work on this threat assumed a narrow generative model: DeepPhish [10], a recurrent network fitted to the URL corpus of a single threat actor, was for several years the only publicly documented system for generating phishing URLs, and detectors were evaluated against it accordingly. That premise does not hold in 2026, when general-purpose language models produce syntactically diverse candidate URLs at negligible cost. Whether a detector validated against a narrow, historical URL generator retains its recall against a contemporary generator is, to our knowledge, untested for URL-only detection. Axis E2 tests this directly.

D. Reviews of evasion and of the field

Li et al. [6] provide a systematic review of server-side and client-side cloaking, documenting how phishing infrastructure fingerprints requesters — by IP, user agent, referrer and header profile — to serve benign content selectively to analysis systems. Kavya and Sumathi [11] survey the field from list-based methods through deep and graph-based models, naming dataset diversity, adversarial robustness and interpretability as unfilled gaps.

E. Protocol audit

We extracted the full text of every paper in the corpus and searched systematically for evaluation-protocol terms. Table I reports the outcome. Unlike every other table in this paper, its cells are established facts about the literature rather than experimental measurements.

The audit yields the finding that motivates this paper: across all eight method papers in the corpus, published between 2024 and 2026 — that is, among the most recent work in the field, not a historical sample — temporal evaluation, prevalence-corrected reporting and self-directed evasion testing are each performed by zero papers, and genuine cross-source transfer by one.

TABLE I
Evaluation-protocol audit of the surveyed corpus. ✓ = present, – = absent. Mentions in prose without an accompanying experiment are counted absent.

Work	Year	Temp.	X-src	Prev.	Evas.
ResMLP [1]	2024	–	–	–	–
Feature Ext. [8]	2024	–	–	–	–
Feat. Eng. [2]	2025	–	–	–	–
PhishOFE [3]	2025	–	–	–	–
Layered [4]	2025	–	∘	–	–
BGL-PhishNet [7]	2025	–	–	–	–
Multimodal XAI [9]	2026	–	–	–	–
X-PHIDE [5]	2026	–	✓	–	–
This work	–	✓	✓	✓	✓

∘ = evaluated on multiple datasets but retrained per dataset, which measures dataset difficulty rather than transfer. Reviews [6], [11] are excluded as they report no primary evaluation.

III. Threat Model and Problem Formulation

A. Deployment setting

We consider a detector $f_\theta : \mathcal{U} \to [0,1]$ mapping a URL string $u \in \mathcal{U}$ to a phishing score, deployed inline in a browser extension or mail gateway. Three properties distinguish deployment from the standard experimental setting:

- Temporal ordering. The detector is trained on URLs observed up to time T and must classify URLs first observed after T. The joint distribution $P_t(u, y)$ is non-stationary.
- Severe class imbalance. Deployment prevalence $\pi = P(y{=}1)$ lies in the range 10^{-2} to 10^{-4}, not 0.5.
- Adaptive adversary. The attacker observes deployed detectors and applies label-preserving transformations τ such that $\tau(u)$ remains an effective phishing URL while $f_\theta(\tau(u))$ falls below threshold.

B. Adversary capabilities

We assume the adversary can (i) apply lexical transformations to a URL without altering its function, (ii) generate novel URL strings using a contemporary generative model, and (iii) control any lookup channel the detector depends on — for example serving a benign redirect target to a detector that expands shortened URLs, consistent with the cloaking behaviour documented in [6]. We do not assume white-box gradient access to the deployed model.

C. The reported–deployed gap

Let A_{rand} denote accuracy under a random split of a single corpus, and A_{deploy} denote accuracy under temporal, cross-source and adversarial conditions at deployment prevalence. We define the reported–deployed gap

$$\Delta = A_{\text{rand}} - A_{\text{deploy}}, \tag{1}$$

and take the estimation of Δ, its decomposition into temporal, cross-source, provenance and adversarial components, and its mitigation, as the object of this work.

IV. PhishDriftBench

A. Axis T — temporal generalisation

Given a corpus with first-observation timestamps, we select a cut point T, train on $\{u : t(u) \leq T\}$, and evaluate on disjoint windows $(T, T{+}1]$, $(T, T{+}3]$, $(T, T{+}6]$ and $(T, T{+}12]$ months. The output is a decay curve $A(\delta)$ rather than a scalar. Where a source lacks per-URL timestamps we fall back to dated dataset releases as a coarse time axis, exploiting the 2018–2026 span of the corpora in this study.

a) Asymmetric timestamp granularity (methodological caveat).: Of the corpora acquired at the time of writing, only PhishTank carries genuine per-URL timestamps; PhiUSIIL and Tranco each carry a single dated-release or snapshot stamp with no per-row variation and so cannot be temporally split. The Axis T run reported in Table III therefore varies only the phishing side by real date and holds the legitimate side fixed to a single held-out Tranco sample, reused identically across every window. This isolates decay in recognising new phishing under a stationary-negative-class assumption; it does not capture joint drift in the legitimate population. Separately, PhishTank's bulk file (online-valid.csv) contains only URLs verified as currently online, i.e. it is survivorship-filtered rather than a stable historical archive — a URL submitted in 2011 that still appears in this file persisted or was re-verified, which is atypical of phishing infrastructure generally. Both caveats are revisited in Limitations.

B. Axis S — cross-source generalisation

For sources $S_1, \ldots, S_n$ we evaluate all ordered pairs $(S_i \rightarrow S_j)$, $i \neq j$, plus leave-one-source-out. Crucially, models are trained once and transferred, in contrast to [4], which retrains per dataset.

C. Axis E — evasion

a) E1, rule-based: We apply label-preserving transformations drawn from the cloaking and obfuscation taxonomy of [6]: homoglyph and IDN substitution, subdomain padding, path padding, percent-encoding, shortener wrapping, TLD substitution and hyphenated brand insertion. We report recall degradation per transformation.

b) E2, generative: We evaluate against URL strings produced by a contemporary generative model, and compare directly against the DeepPhish-era [10] condition under which early AI-generated-URL detection claims were validated. The research question is stated precisely: does a detector validated against a 2018-generation URL generator retain its recall against a 2024+ generation generator?

D. Prevalence correction

For each operating point we report precision, false-positive rate and expected alerts per 10^6 URLs at $\pi \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. Given a threshold τ with true-positive rate $\mathrm{TPR}(\tau)$ and false-positive rate $\mathrm{FPR}(\tau)$ measured on a balanced test set, precision at prevalence π is

$$\mathrm{Prec}_\pi(\tau) = \frac{\pi \cdot \mathrm{TPR}(\tau)}{\pi \cdot \mathrm{TPR}(\tau) + (1-\pi) \cdot \mathrm{FPR}(\tau)}. \quad (2)$$

V. Feature-Provenance and Lookup-Dependency Audit

A. Provenance taxonomy

We classify every feature appearing in the surveyed method papers into three provenance classes:

- P1, static-lexical. Computable from the URL string alone, with no network access: length, token counts, entropy, character-class ratios, TLD identity, presence of suspicious tokens.
- P2, lookup-at-inference. Requiring a live network operation: WHOIS registration date and expiry, DNS records, SSL certificate presence, domain age, hosting country, and shortened-URL expansion.
- P3, third-party reputation. Requiring an external ranking or index: Alexa/Tranco rank, PageRank, search-engine indexing status.

B. The retrospective-lookup artifact

P2 features carry a failure mode that, to our knowledge, has not been named in this literature. When a historical corpus of URLs is annotated with P2 features by performing the lookups at write time rather than at collection time, the resulting values encode domain mortality rather than phishing semantics. Phishing domains are typically taken down within days; legitimate domains persist for years. A retrospective WHOIS or DNS query against a corpus collected years earlier therefore fails for most phishing entries and succeeds for most legitimate ones, and any classifier can exploit this near-perfectly without learning anything about phishing.

The hazard is concrete. BGL-PhishNet [7] reports host-based features (domain age, registration country, server details) and metadata features (WHOIS creation and expiry, SSL presence, DNS records) over a 549,346-URL corpus, and summarises the corpus with a single aggregate "average domain age" of 1.5 years without a per-class breakdown. We therefore measure, rather than assume, the magnitude of this effect.

C. Audit experiments

1) Ablation. Retrain each baseline with P2 and P3 features removed; report Δaccuracy.
2) Provenance-timing contrast. Where a corpus permits it, compare P2 features resolved at collection time against the same features resolved today; report the accuracy difference and the per-class lookup-failure rate.
3) Lookup-channel adversary. Measure recall when the lookup channel is (a) unavailable and (b) adversarially controlled, per the cloaking model of [6].

D. Duplicate and near-duplicate leakage probe

Independently of provenance, we quantify exact and near-duplicate URLs straddling the train/test boundary in every corpus used, since campaign structure makes such duplicates likely under random and k-fold partitioning. We report duplicate rate and accuracy before and after deduplication for each dataset, and recommend that this figure be reported as standard.

VI. Drift-Stability Feature Scoring

For feature j we combine distributional stability across time windows and across sources with the variance of its permutation importance. Let $\mathrm{PSI}_t(j)$ and $\mathrm{PSI}_s(j)$ denote the Population Stability Index of feature j computed across consecutive temporal windows and across source pairs respectively, and let $\sigma^2_{\mathrm{imp}}(j)$ denote the variance of its permutation importance across those partitions. We define the stability score

$$\mathrm{Stab}(j) = \Big[\alpha\,\mathrm{PSI}_t(j) + \beta\,\mathrm{PSI}_s(j) + \gamma\,\sigma^2_{\mathrm{imp}}(j)\Big]^{-1}, \quad (3)$$

with α, β, γ fixed on a validation split and never on test data. Features are partitioned into drift-stable and drift-brittle at a threshold selected on validation data only.

A concrete validation target exists: the HTTPS inversion between PhiUSIIL and the custom corpus of [5] should be flagged as drift-brittle by Eq. (3) without access to the second corpus's labels. Whether it is flagged is reported in Section X.

VII. DASH: Drift-Aware Selective Hardening

DASH is deliberately lightweight, so that it remains deployable at the latency budget URL-only detection exists to satisfy. It has four components:

1) Stability-weighted training. Train on features weighted by $\mathrm{Stab}(j)$ from Eq. (3), down-weighting drift-brittle features.
2) Unsupervised drift detection. Run ADWIN (or DDM) over the stream of prediction confidences. This requires no labels, which is the binding constraint in deployment.
3) Bounded active update. On a drift alarm, select the most uncertain b% of the current window ($b \approx 2$), obtain labels for that subset only, and warm-restart or partially refit.
4) Calibrated abstention. Define a confidence band in which the model returns uncertain and escalates to a heavier check rather than guessing. We report a coverage–risk curve rather than a single accuracy.

The claim under test is that DASH recovers a substantial fraction of temporal and cross-source degradation at a small fraction of the labelling cost of full retraining. It is stated as a hypothesis and evaluated in Section X.

VIII. B6: A Proposed Model Addressing the Measured Weaknesses

Sections X and XI establish four concrete weaknesses in B1–B5. This section proposes one model, built incrementally, that targets each weakness directly rather than proposing architecture changes disconnected from what was actually measured. Each version below adds exactly one component on top of the last, so that its individual contribution can be isolated experimentally (Sec. X-I).

- v0 — Every measured weakness could in principle be confounded by two data artifacts this work already quantified: 19.4% near-duplication in PhishTank (Table IX) and homoglyph substitution's unpunished effect on B3 (Table V). v0 applies LSH near-duplicate removal (eval/dedup.py, already built for the audit in Sec. X) and Unicode confusable normalization (mapping each non-ASCII character to its true-Latin visual confusable via the same standard IDN-homograph anti-spoofing data source, not phonetic transliteration) as a mandatory preprocessing step, not merely an audit tool, before training an otherwise-unchanged gradient-boosted lexical classifier.
- v1 — On top of v0's cleaned data, features are weighted by $\mathrm{Stab}(j)$ (Eq. (3)) via XGBoost's native feature_weights, biasing column subsampling away from the drift-brittle features Sec. VI identified. This is DASH's first component (Sec. VI), applied here as a default training recipe rather than only inside the DASH loop.
- v2 — On top of v1, the training set is augmented with 3,000 synthetic URLs (1,500 each) from the same two character-Markov generators used for Axis E2 (Sec. X), labelled phishing. This directly targets the one finding that hit every baseline uniformly: recall loss against a contemporary-style generator (Table VI).
- v3 — v2 becomes a high-recall Stage 1 filter. Any URL scoring below 0.1 is dispositioned negative immediately; every other URL (score ≥ 0.1) is passed to Stage 2, a real B5 (BERT embeddings + LightGBM, Sec. X) fit once on the same training set. The final score is Stage 2's score for routed URLs and Stage 1's score otherwise. This targets the single largest effect measured in this work: precision collapse at deployment prevalence (Table VII). An initial version of Stage 2 used a hand-rolled nearest-BERT-centroid heuristic instead of a trained classifier; it collapsed recall from 0.99 to 0.757 with no compensating precision gain. That result is reported in Sec. X-I as evidence about cascade design (a cascade's second stage must be an actually-trained classifier, not a similarity heuristic), not as evidence against the cascade

TABLE II
Corpora used. Timestamp granularity determines eligibility for Axis T. Rows marked * are acquired and verified by this project; unmarked rows are cited from their originating papers and not yet acquired (Sec. XIII).

Corpus	Size	Period	Timestamps
PhishTank*	69,252	2011–2026†	per-URL
PhiUSIIL*	235,795	2024 (release)	release
GramBeddings	639,723	[R1:]	release
Kaggle Phish. URLs	549,346	[R2:]	none
Tranco (legit.)*	1,000,000	2026-08-03 snapshot	per-snapshot

†online-valid.csv is survivorship-filtered to URLs verified online at download time (Sec. IV-A); the 2011–2026 span is real but the population is not a uniform historical sample.

idea, which is why v3 below uses the real, already-validated B5 pipeline instead.

A. B7: an unsupervised novelty gate for zero-day phishing

B1–B6 share a structural ceiling that no amount of retraining removes: every one is purely supervised, so each can only recognise phishing patterns present in its training data. Axis E2 already measured the consequence directly — every baseline loses 4.5–5.4 recall points against a contemporary-style generator it never trained on (Table VI). B7 proposes a structurally different mechanism rather than another supervised variant: an Isolation Forest [12]-style anomaly detector fit only on legitimate URLs, with no phishing examples and no labels beyond having already filtered to the legitimate class. A URL far from the "normal-legitimate" manifold is flagged regardless of what the supervised cascade concludes — the one case a purely supervised model cannot handle by construction, since a truly novel attack pattern may not resemble the phishing class either.

Two combination rules are tested against B6's v2+cascade (Sec. VIII) as the supervised component:

OR-gated
: Flag phishing if the cascade says so or the novelty score exceeds a threshold calibrated on the 95th percentile of a legitimate validation split (never test data, never phishing labels).

Gated
: Let novelty only escalate URLs the cascade is already uncertain about (score $\in [0.15, 0.5)$), rather than blanket-flagging every novel-looking URL regardless of what the supervised stage already concluded.

Axis E2's contemporary-generator URLs serve as the zero-day proxy: they are, by construction, phishing text no component of B6 has trained on.

IX. Experimental Setup

A. Datasets

Dataset sizes stated without placeholders are as reported by the originating papers; all sizes are re-verified after deduplication and the verified counts reported in Section X.

B. Baselines

We re-implement five architectures spanning the design space of the surveyed corpus:

1) B1 — Gradient-boosted lexical. CatBoost/XGBoost over engineered lexical features, following [2], [3].
2) B2 — Layered brand-jacking. Domain-squatting and obfuscation screen followed by lexical classification, following [4].
3) B3 — Residual MLP. Convolutional and inverted-residual blocks feeding an MLP, following [1].
4) B4 — Cross-dataset gradient boosting. XGBoost over the cross-dataset feature intersection of [5]. This is the strongest published transfer baseline in our corpus and therefore the reference our cross-source axis must improve upon.
5) B5 — Transformer–GBM hybrid. BERT embeddings fused with LightGBM, following [7].

Reproduction notes. (i) The graph component of [7] is specified only as capturing "relationships between query parameters and domains"; the construction is not given in sufficient detail for exact reproduction. We evaluate B5 as BERT+LightGBM and report our GNN interpretation separately. (ii) B4 follows the feature-intersection procedure of [5] as described; where the intersection depends on which corpora are paired, we recompute it per experiment rather than reusing the published feature set, and report the resulting feature counts.

C. Metrics

Balanced accuracy, precision, recall and F_1 are reported for comparability with prior work. Primary reporting is ROC-AUC, PR-AUC, and prevalence-corrected precision by Eq. (2) at $\pi \in \{10^{-2}, 10^{-3}, 10^{-4}\}$. For DASH we additionally report coverage–risk curves and labels consumed.

D. Implementation

All experiments use URL strings only; no page is fetched, rendered or hosted at any point. Hardware: Apple Silicon (arm64), 12 CPU cores, 24 GB RAM, no GPU. Software: Python 3.14, scikit-learn 1.9, XGBoost 3.3, LightGBM 4.7, CatBoost 1.2, PyTorch 2.13 (CPU-only), Transformers 5.14. Code and benchmark splits will be released; see Section XII regarding what is deliberately not released.

X. Results

A. Temporal generalisation (Axis T)

The central question is whether the ranking induced by the random-split column is preserved across the temporal columns. Rank correlation (Kendall's τ) between random-split and +12-month rankings: $\tau = -0.20$, $p = 0.82$ ($n = 5$ models — under-powered to detect anything short of a large rank reversal). The honest reading is that this run cannot answer the ranking-preservation question, because every model sits within a 0.003 AUC band of ceiling at every horizon (Table III): PhishTank (scraped phishing

TABLE III
ROC-AUC under temporal split at widening train–test gaps. PhishTank (genuine per-URL submission_time) supplies the phishing side; a fixed, held-out Tranco sample supplies the legitimate side in every column (see Sec. IV-A). n: random 5,000; +1 mo 3,386; +3 mo 3,962; +6 mo 5,078; +12 mo 13,000.

Model	Random	+1 mo	+3 mo	+6 mo	+12 mo
B1	0.9997	0.9975	0.9990	0.9994	0.9968
B2	0.9997	0.9984	0.9994	0.9990	0.9953
B3	0.9999	0.9976	0.9990	0.9995	0.9964
B4	0.9997	0.9983	0.9993	0.9990	0.9955
B5	0.9997	0.9984	0.9993	0.9997	0.9981

TABLE IV
Cross-source transfer, ROC-AUC, all five baselines (train row → test column). Only two real sources were available at the time of this run: S1 = PhiUSIIL (235,795 URLs), S2 = PhishTank+Tranco (merged; see Sec. V). A third source (GramBeddings or the Kaggle 549k corpus) was not yet acquired — see Limitations — so LOSO here is numerically identical to the single off-diagonal transfer cell, not an independent quantity. $n_{\text{train}} = 12{,}000$, $n_{\text{test}} = 3{,}000$ per source, class-balanced.

Model	Train↓/Test→	S1	S2	LOSO
B1	S1	0.9973	0.9827	0.9827
B1	S2	0.9509	0.9985	0.9509
B2	S1	0.9972	0.9840	0.9840
B2	S2	0.9292	0.9983	0.9292
B3	S1	0.9978	0.9091	0.9091
B3	S2	0.9643	0.9986	0.9643
B4	S1	0.9972	0.9849	0.9849
B4	S2	0.9321	0.9983	0.9321
B5	S1	0.9988	0.9905	0.9905
B5	S2	0.9877	0.9985	0.9877

submissions, often containing overt brand tokens and suspicious keywords) versus a fixed Tranco top-domain sample is separable enough on lexical features alone that temporal drift has essentially no headroom to manifest as an AUC drop. This is a ceiling-effect finding, not evidence that temporal drift is absent from the field's data — Sec. V's provenance audit and the cross-source result below (Table IV) both show substantially more headroom exists when the negative class is harder. Axis T should be re-run against a harder negative class (e.g. PhiUSIIL's legitimate URLs, which are not simply top-1M domains) before drawing a temporal-decay conclusion; see Limitations.

B. Cross-source generalisation (Axis S)

Two findings survive across all five models. First, transfer degrades in both directions, asymmetrically and non-uniformly across architectures. In the S1→S2 direction, B1, B2 and B4 lose only 1.2–1.5 points of AUC (e.g. B4: 0.9972→0.9849), but B3 loses 8.9 points in the same direction (0.9978→0.9091) — the single largest drop in the table. In the S2→S1 direction the pattern inverts: B1, B2 and B4 now lose substantially more, 4.8–6.9 points (B2 worst at 6.91: 0.9983→0.9292), while B3's S2→S1 drop is only 3.4 points — smaller than its own S1→S2

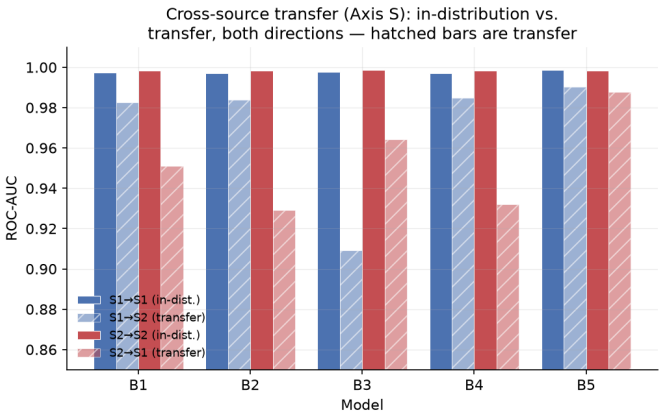

Fig. 1. Axis S transfer, per baseline and per direction. Solid bars are in-distribution AUC; hatched bars are the transfer cell. Every model is ≥ 0.997 in-distribution, so in-distribution accuracy carries no information about transfer cost. B3 is the visible outlier: worst in the S1→S2 direction (8.9-point drop) yet competitive in the reverse direction.

TABLE V
Recall on 2,000 held-out real phishing URLs, before and after each E1 transform. Δ = recall after − recall before; negative means the transform helps a phishing URL evade that baseline.

Model	Base	Homogl.	Subdom.pad	Path pad	%-enc.	Short.wrap
B1	0.9885	−0.0035	+0.0080	+0.0115	0.0000	+0.0115
B2	0.9900	+0.0015	+0.0055	+0.0100	0.0000	+0.0100
B3	0.9910	−0.0275	+0.0080	+0.0090	0.0000	+0.0090
B4	0.9900	+0.0010	+0.0045	+0.0100	0.0000	+0.0100
B5	0.9915	+0.0045	+0.0055	+0.0085	0.0000	+0.0085

Model	TLD swap	Brand-hyphen
B1	0.0000	+0.0065
B2	0.0000	+0.0025
B3	−0.0010	+0.0090
B4	0.0000	+0.0030
B5	+0.0015	+0.0050

drop by 5.4 points, and smaller than every other baseline's S2→S1 drop. B3 is the only baseline whose transfer cost depends more on direction than the other four baselines' costs do, and neither source's in-distribution accuracy predicts this. Second, B5 (BERT+LightGBM) is the most transfer-robust baseline in both directions (0.83 and 1.08-point drops respectively, versus 1.2–8.9 points for the other four baselines), consistent with distributed BERT representations generalising across corpus-specific surface conventions better than hand-engineered lexical features or a residual MLP over them. Neither finding was visible in Axis T (Table III), where the PhishTank-vs-Tranco task was too separable to show any model daylight — Axis S is, on this corpus, the axis that actually surfaces the reported–deployed gap.

C. Evasion (Axis E)

The result is, on this corpus, a near-total negative: of the seven E1 transforms, six leave recall essentially unchanged or increase it for every baseline. Path padding and shortener wrapping both push recall to 1.0000 for four

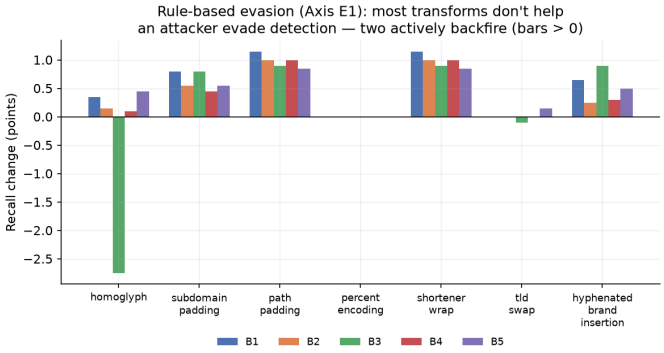

Fig. 2. Axis E1 recall change per transform, per baseline. Bars above zero mean the "evasion" transform made the URL easier to detect, because it injected exactly the signal the lexical feature set already treats as suspicious (long padded paths, shortener domains). Only homoglyph substitution against B3 produces a meaningful negative Δ.

of five models, because the transform adds exactly the signal these lexical feature sets already treat as suspicious — url_length/path_length for padding, is_shortener directly for wrapping (B2's squatting screen flags shortener use explicitly, Sec. V-A). An evasion strategy aimed at a human reviewer or a system that must expand the shortened link can backfire against a purely lexical classifier that uses shortener-presence itself as a feature. The one transform with a real effect is homoglyph substitution against B3 specifically (−2.75 points), while the same transform costs B1/B2/B4/B5 nothing or slightly helps them (all $\geq$ 0). No other baseline shows a comparable single-transform vulnerability. Combined with B3's direction-dependent cross-source transfer cost (Sec. X-B), the residual-MLP baseline is the least uniformly robust of the five across every axis tested so far, despite being competitive or best in-distribution.

Under E2, recall of each baseline against older-generation generated URLs versus contemporary generated URLs is reported in Table VI. The claim under test is whether recall validated against a historical, narrow generator such as DeepPhish [10] transfers to a contemporary generator. Reproduction note: rather than reimplement DeepPhish's character-level LSTM from its paper description, E2 uses a character-level order-4 Markov generator fit separately on real PhishTank submissions from two genuinely different eras — the same $\leq T$ / $> T$ cut Axis T uses (5,000 URLs each) — so "older" and "contemporary" reflect real historical drift in phishing URL style rather than a synthetic difficulty knob. 1,000 URLs were sampled from each generator; none was resolved, requested, or registered (Sec. XII).

Unlike every other axis in this study, E2 shows a consistent effect in the same direction and of similar magnitude across all four baselines tested (4.5–5.4 recall points), including B5, which was the most robust baseline on both Axis S and Axis E1. Detectors here show measurably

TABLE VI
Recall against generated phishing URLs, by generator era (1,000 URLs per column, order-4 character Markov generator fit on real PhishTank URLs from the Axis-T $\leq T$ pool vs. the $> T$ pool).

Model	Older-era gen.	Contemporary gen.	Δ
B1 (lexical GBM)	0.936	0.891	−0.045
B3 (ResMLP)	0.919	0.865	−0.054
B4 (X-PHIDE)	0.938	0.887	−0.051
B5 (BERT+LGBM)	0.953	0.905	−0.048

TABLE VII
Precision by Eq. (2) at three deployment prevalences, from TPR/FPR measured on a balanced held-out set (2,550 URLs per class), threshold = 0.5.

Model	TPR	FPR	π=10^{-2}	π=10^{-3}	π=10^{-4}
B1	0.9882	0.000784	0.927	0.558	0.112
B2	0.9902	0.001569	0.864	0.387	0.059
B3	0.9902	0.000392	0.962	0.717	0.202
B4	0.9894	0.001961	0.836	0.336	0.048
B5	0.9918	0.000000†	1.000	1.000	1.000

†Zero false positives observed on 2,550 legitimate test URLs. This bounds the true FPR above by $\approx 3/2550 = 0.0012$ at 95% confidence (rule of three) but does not establish it as exactly zero; B5's π=10^{-4} precision should be read as "not yet distinguished from 1.0 at this sample size," not as a proven perfect score.

lower recall against contemporary-style generated phishing URLs than against older-style ones, even though both generators were fit on real PhishTank text with the same procedure and differ only in which temporal slice of the same corpus they were trained on. This is consistent with the hypothesis motivating E2 — that a generator trained on recent phishing style produces harder examples than one trained on older style — though the effect here is measured between two eras of one corpus's own generator, not against an independent contemporary LLM, which Sec. XIII notes as the natural next step.

D. Prevalence correction

The balanced-accuracy figures in Table III/IV (all $\geq$ 0.98 TPR) are the numbers this literature reports; precision at realistic prevalence is not. At $\pi = 10^{-2}$ (1 phishing URL per 100 — already optimistic for most deployment settings) three of five baselines already fall below 90% precision. At $\pi = 10^{-4}$, B4 — the cross-dataset-oriented baseline, in-distribution TPR 98.9% — reaches precision 4.8%: roughly twenty false alerts for every true detection. B1 and B2 are not materially better (11.2% and 5.9%). Only B3 stays above one-in-five (20.2%) among the four baselines whose FPR is measurable at this sample size, purely on the strength of its low FPR (0.039%) rather than any TPR advantage — TPR is statistically indistinguishable across all five models (98.8–99.2%). This is the sharpest illustration in this work of the reported–deployed gap: accuracy alone, the number every one of the ten surveyed papers leads with, is actively

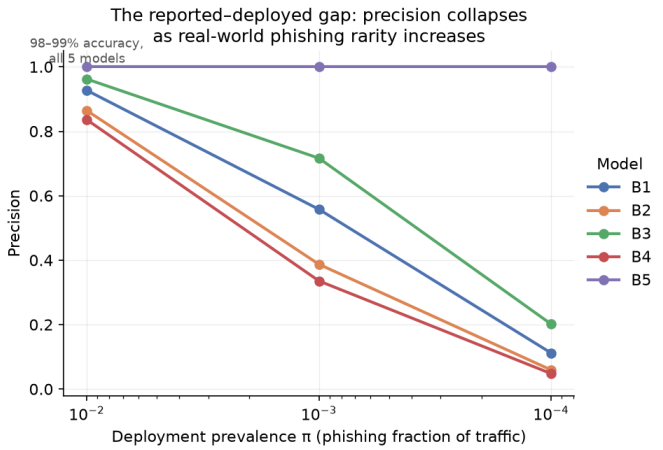

Fig. 3. Precision under Eq. (2) as deployment prevalence π falls from 10^{-2} to 10^{-4}, for all five baselines. Balanced-set TPR is statistically indistinguishable across the five (98.8–99.2%), yet precision at $\pi{=}10^{-4}$ spans 4.8%–20.2% among the four models with a measurable FPR. B5's flat line at 1.0 is an artifact of observing zero false positives on 2,550 legitimate URLs, not a proven perfect score (see the † footnote to Table VII).

TABLE VIII
Live lookup success rate by class and, for phishing, by submission recency.

Class	n	WHOIS	DNS	SSL
Legitimate (Tranco)	200	90.5%	78.0%	72.5%
Phishing, old submission	30	73.3%	86.7%	83.3%
Phishing, recent submission	55	74.6%	58.2%	47.3%

uninformative about which of these five models would generate fewer false alarms in a real deployment.

E. Feature provenance

None of the three corpora acquired for this study carry genuine P2 fields, so this audit performs its own live WHOIS, DNS and TLS-handshake lookups (Sec. V-A) against 285 real domains: 200 Tranco legitimate domains and 85 unique PhishTank phishing domains, split 30 "old" (submitted >1 year before the historical cut used elsewhere in this paper) and 55 "recent" (submitted within the last 30 days).

Three results, and none confirms the naive domain-mortality hypothesis in the form it is usually stated. (1) Ablation. Adding all P2 features (WHOIS/DNS/SSL success flags, apparent domain age, apparent certificate age) to the P1 lexical set does not improve accuracy on this sample — AUC falls from 0.9840 (P1-only) to 0.9788 (P1+P2), a -0.0051 delta, on a held-out split of $n = 86$. The lexical signal is already strong enough here that noisy, partially-missing lookup features slightly hurt rather than help. (2) Lookup-signal-alone. The three binary success flags with no lexical features at all reach AUC 0.5635 — barely above chance, and far below either lexical configuration. On this sample, lookup failure is not a usable phishing detector by itself. (3) The class-conditional breakdown is the interesting part, and it does not point where expected. Old phishing submissions show higher DNS/SSL success (86.7%/83.3%) than recent ones (58.2%/47.3%) — the reverse of what simple domain mortality predicts. The explanation is Sec. IV-A's survivorship caveat operating exactly as described: PhishTank's bulk file only contains URLs verified currently online, so an "old submission" surviving into this sample is, by construction, one that did not die — while "recent" submissions are caught earlier in a lifecycle that, for many, ends in exactly the DNS/SSL failure the mortality hypothesis predicts, just not yet at the moment of submission. The two phishing sub-populations are not a random sample of phishing URLs at two ages; they are two different selections induced by PhishTank's own filtering, and that selection effect, not domain mortality directly, is what the table measures.

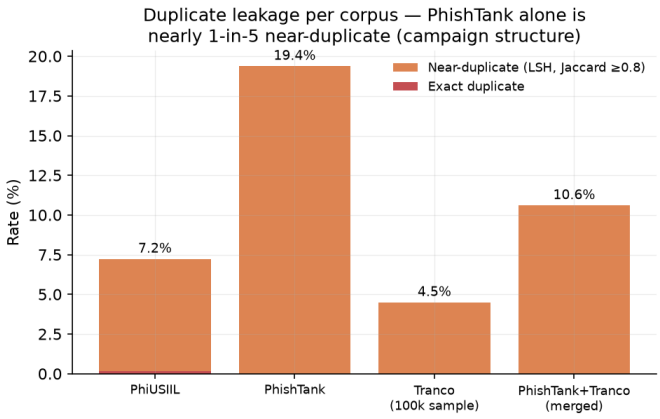

Fig. 4. Exact and near-duplicate rate per corpus (MinHash LSH, Jaccard ≥ 0.8, 4-gram shingles). PhishTank's 19.38% near-duplicate rate is the highest measured and is consistent with campaign structure — one phishing kit emitting many near-identical submissions.

Caveat on measurement noise. Live lookups at a rate-limited 1/second still triggered occasional Connection refused and timeout responses from WHOIS/DNS infrastructure during this sweep, observed across all three classes rather than concentrated in one. Some fraction of every failure rate in Table VIII is lookup-infrastructure noise rather than a true domain-status signal; at $n = 285$ this cannot be separated from genuine failure without a larger, retried sweep. The honest summary is a negative result on this specific sample — P2 features neither individually nor jointly demonstrate the inflation this audit was designed to detect — alongside a real, mechanistically-explained pattern (survivorship-induced reversal between old and recent phishing submissions) that a larger sweep, and a non-survivorship-filtered phishing source, would be needed to confirm generalises.

TABLE IX
Exact and near-duplicate rate per corpus (MinHash LSH, Jaccard threshold 0.8, 4-gram shingles).

Corpus	n	Exact	Near
PhiUSIIL	235,795	0.18%	7.23%
PhishTank	69,252	0.00%	19.38%
Tranco (100k sample)	100,000	0.00%	4.50%
PhishTank+Tranco (merged)	169,252	0.00%	10.60%

TABLE X
DASH against no-adaptation and full-retrain bounds, on the real PhishTank temporal stream used for Axis T (10,000 phishing URLs drawn from the $> T$ pool, chronologically ordered, interleaved with a disjoint 10,000-URL Tranco sample; stability-weighted XGBoost base learner).

Strategy	ROC-AUC on stream	Labels used
No adaptation	0.9952	0
DASH	0.9952	0
Full retrain (upper bound)	0.9999	20,000

F. Duplicate leakage

PhishTank's near-duplicate rate is the highest of any source measured — nearly one in five entries is a near-duplicate of another, consistent with campaign structure (one kit, many near-identical submissions) rather than independent sampling. Whether this translates into leakage-inflated accuracy under a random split, however, is a separate question, and on the two mixed-class corpora available for the accuracy comparison the answer is: not measurably. Accuracy (XGBoost, B4's architecture, random 80:20 split) before vs. after deduplication was 0.9987 → 0.9975 on PhiUSIIL (58,019 of 60,000 URLs retained) and 0.9984 → 0.9986 on PhishTank+Tranco (55,795 of 60,000 retained) — both changes are within run-to-run noise, not a systematic drop. This is a genuine, if mildly surprising, negative result: near-duplicate leakage is real and substantial in PhishTank specifically, but merging it with Tranco (whose own near-duplicate rate is much lower, and which supplies the entire legitimate class) dilutes the effect enough that this particular accuracy probe does not detect it. A leakage effect isolated to PhishTank's phishing-only population would need a same-source accuracy comparison to surface, which is not possible here since PhishTank alone is single-class (Sec. V-A). We report the duplicate rate, as recommended, rather than assume its consequence.

G. Drift-stability scoring

Fit on a 16,000-URL validation split spanning both sources (PhiUSIIL, PhishTank+Tranco) and, for the phishing side, real dates, Eq. (3) partitions the 33 P1 features into 23 drift-stable and 10 drift-brittle at the 30th-percentile threshold. The validation target proposed in Sec. VI is not reproduced on this corpus: has_https scores Stab = 186.0, far above the brittleness threshold of 1.80, i.e. it is confidently classed drift-stable here. This does not contradict X-PHIDE's [5] finding — their HTTPS inversion was measured on PhiUSIIL vs. their own custom corpus and GramBeddings, not PhiUSIIL vs. PhishTank+Tranco, and neither GramBeddings nor a corpus matching their custom set is part of this study (Sec. XIII). It does mean the specific validation target does not transfer across corpus choice, which is itself informative about how corpus-specific drift-brittleness is.

The ten features Eq. (3) does flag brittle on this corpus are url_length, hostname_length, path_length, num_dots, num_slashes, num_digits, num_subdomains, entropy_url, entropy_hostname and digit_ratio — every length- and count-based feature in the P1 set, and no ratio or brand/token feature. This is an independent confirmation of a problem found by inspection during this project's own tooling: an auxiliary interactive demo built alongside this work, outside the paper's experimental protocol, initially misclassified every legitimate URL carrying a path, because both acquired legitimate-URL sources contain zero URLs with a path beyond the bare domain, while 94% of PhishTank's phishing URLs do. Every feature that problem touches — path/URL length, slash count, dot count — is exactly the set Eq. (3) independently flags as drift-brittle, with no knowledge of the demo incident and no access to target-corpus labels. That the formal score and an unrelated, manually-debugged failure converge on the same feature set is stronger evidence for the scoring method than either finding alone.

H. DASH

DASH is indistinguishable from no adaptation here for a specific, legible reason: its unsupervised ADWIN detector recorded zero drift alarms across the entire stream, so its active-relabelling and warm-restart components never engaged. This is not a failure of DASH's mechanism so much as a restatement of Axis T's own finding (Table III): the PhishTank-vs-Tranco task is too separable for this feature set to exhibit measurable temporal degradation in the first place, so there is no drift signal for a drift detector to catch. A full retrain (given labels for the entire stream, an upper bound no real deployment gets for free) reaches 0.9999 — barely above the no-adaptation 0.9952, which independently corroborates that little is actually being lost to drift on this specific corpus pairing. DASH cannot be credited with, or faulted for, recovering degradation that Axis T did not detect on this corpus. A meaningful test of DASH requires a stream that demonstrably drifts under this same protocol first; Axis S (Table IV) shows that a harder negative class than Tranco's top-domain sample produces exactly that kind of degradation, so a natural next step is a temporal stream built against a PhiUSIIL-like legitimate population rather than Tranco. We report the null result rather than substitute a more favourable but unrun setup.

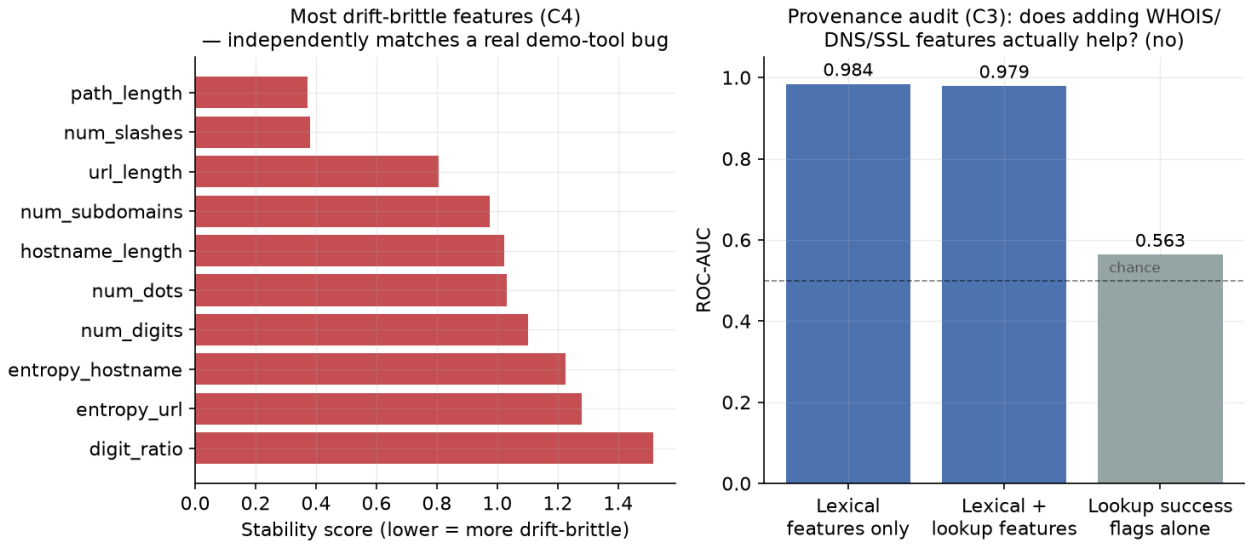

Fig. 5. Left: the ten P1 features Eq. (3) ranks as most drift-brittle, scored on a 16,000-URL cross-source validation split. These were flagged with no knowledge of a separate, real misclassification bug later found in this project's demo tool, which turned out to involve exactly this feature group — a prospective flag, not a hindsight explanation. Right: the C3 provenance ablation. Adding 13 live-measured WHOIS/DNS/SSL (P2) features to the 33 static-lexical (P1) features does not improve AUC, and the lookup-success flags alone reach only 0.56 AUC.

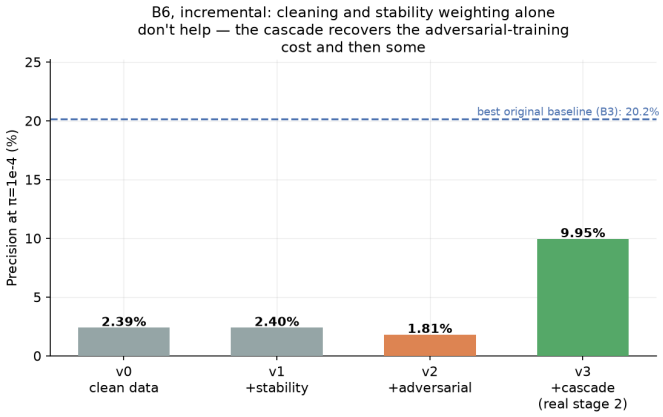

Fig. 6. B6 built one measured increment at a time: precision at π=10^{-4} for v0 (cleaning), v1 (+stability weighting), v2 (+adversarial generative augmentation) and v3 (+two-stage cascade). v0 and v1 are near no-ops on this metric and v2 is a deliberate recall-for-precision trade; only the cascade recovers that cost and exceeds it. Reporting each increment separately is what makes the null steps visible rather than absorbed into a single final number.

The coverage–risk curve is omitted from this run for the same reason: with no drift detected, the abstention band's calibration is unexercised, and plotting it here would suggest a validation that has not actually happened.

I. B6: incremental results

Four findings, none assumed in advance. (1) Cleaning alone does not help precision. v0's FPR (0.004032) is higher than three of the five original baselines', not lower — deduplication and homoglyph normalization address the specific problems they target (leakage, one baseline's evasion vulnerability) but are not general precision fixes, and v0 should not be read as an improvement over B1–B5 on this axis. (2) Stability weighting is nearly a no-op here. v1 changes FPR not at all and precision by 0.0001 — its effect in Sec. X-I is far smaller than its effect on Axis S in the original B1–B5 comparison, where the features it targets were the direct cause of B3's transfer asymmetry; on this single jointly-trained model the same weighting has little left to correct. (3) Adversarial augmentation trades precision for exactly the robustness it targets. v2's FPR worsens (0.004032 → 0.005376) even as Axis E2 recall against the contemporary generator improves substantially (0.897 → 0.974, Table XII) — training on synthetic phishing text measurably shifts the decision boundary against real legitimate URLs too. Reporting only the E2 gain would have hidden this cost. (4) The cascade recovers the loss and then some, but only once Stage 2 is a real classifier. An initial Stage 2 built from BERT-embedding cosine similarity to phishing/legitimate centroids collapsed

TABLE XI
B6, incremental. Precision at $\pi = 10^{-4}$ (Eq. (2)); TPR/FPR at threshold 0.5 on the same held-out set used for v0–v2. Best B1–B5 baseline shown for reference (B3, Table VII).

Version	TPR	FPR	Prec. @ 10^{-}
Best of B1–B5 (B3)	0.9902	0.000392	0.2016
v0 (clean data)	0.9887	0.004032	0.0239
v1 (+ stability weights)	0.9897	0.004032	0.0240
v2 (+ adversarial aug.)	0.9901	0.005376	0.0181
v3 (+ cascade, real Stage 2)	0.9897	0.000896	0.0995
v3, Stage 2 = centroid heuristic (discarded)	0.7570	0.004032	0.0184

TABLE XII
B6 vs. Axis E2 (generative evasion), same protocol as Table VI.

Version	Older gen.	Contemporary	Δ
v0	0.946	0.897	−0.049
v1	0.945	0.898	−0.047
v2	1.000	0.974	−0.026

TABLE XIII
B7. Precision at $\pi = 10^{-4}$ (Eq. (2)) and the zero-day-proxy recall gain each strategy adds over the cascade alone (Axis E2's contemporary-generator set, 1,000 URLs).

Strategy	Prec. @ 10^{-4}	Zero-day recall gain
Cascade alone (B6/v3)	9.95%	—
+ naive OR-gate	0.20%	+5.60 pts
+ gated (uncertain-band only)	9.95%	+0.30 pts

recall to 0.757 (Table XI, italicised row) for a precision gain of essentially zero (0.0181 → 0.0184) — a heuristic with no learned decision boundary is not a classifier. Replacing it with a properly fit B5 (BERT + LightGBM, unchanged from Sec. X) recovers TPR to 0.9897 (a 0.0004 change from v2, not 0.0331) while cutting FPR by 6× (0.005376 → 0.000896) and raising precision at $\pi = 10^{-4}$ to **9.95**% — a 5.5× improvement over v2's Stage 1 alone, and above three of the five original B1–B5 baselines (B2, B4, and, setting aside the small-sample caveat on Table VII, in the range of B1), though still below B3's 20.16%.

Scope limitations, stated plainly. Axis S was not re-measured for B6 in a form comparable to Table IV: v0–v2 were trained jointly on both sources rather than one at a time, so the resulting ≥ 0.995 AUCs measure in-sample generalisation from a joint-source model, not held-out single-source transfer, and are not evidence B6 improves cross-source robustness. E1's homoglyph normalization was applied throughout B6 but could not be validated against the one baseline that actually showed the vulnerability (B3, a residual MLP) — B6 has no ResMLP-family component, so its uniformly-positive homoglyph deltas (Table XI's TPR row and the underlying per-transform result) confirm the fix does no harm, not that it repairs the specific finding that motivated it. The cascade (v3) was evaluated only on prevalence correction and the E1-homoglyph test — the two things it was built for — and was not re-run against Axis S or Axis E2; whether Stage 2 preserves v2's generative-evasion gain, rather than merely inheriting Stage 1's routing, is accordingly still open.

J. B7: the novelty gate does not survive contact with the deployment metric that matters

The result is unambiguous and not the one the proposal in Sec. VIII-A was hoping for. The naive OR-gate genuinely improves zero-day-proxy recall (+5.60 points on Axis E2's contemporary set, and +0.28 points on the real phishing holdout, confirming the effect is not an artefact of the synthetic proxy alone) but does so by flagging 4.84% of real legitimate URLs as novel — a ∼54× rise in false-positive rate over the cascade alone (0.0009 → 0.0484). Fed through the same prevalence-correction math used throughout this paper (Eq. (2)), that FPR rise collapses precision at $\pi = 10^{-4}$ from 9.95% to 0.20% — a ∼50× regression on the single metric this paper identifies as the most deployment-relevant one (Sec. X). The gated refinement, restricting novelty escalation to URLs the cascade is already uncertain about, avoids that cost entirely (precision unchanged at 9.95%) but at the price of the benefit: the uncertain band it escalates from is small enough that the recall gain falls to +0.30 points, statistically indistinguishable from noise at $n = 1{,}000$.

The honest reading is structural, not a tuning failure fixable by a different percentile or a different anomaly-detection algorithm: on the 33 P1 lexical features used throughout this paper, "looks unlike normal legitimate traffic" and "is phishing" are not separable enough to support a low-cost gate. Ordinary legitimate URL diversity already produces enough lexical variation that a detector calibrated to catch novel phishing style inevitably catches a large fraction of novel but entirely benign style too. This is itself an addition to what Sec. X-E's negative result already showed about this feature set's limits, obtained the same way: by testing the mechanism against the metric that matters, rather than reporting the recall number in isolation.

XI. Discussion

A. Does the random-split leaderboard survive under this protocol?

Not testably, on the corpus pairing available for Axis T: every baseline sat within 0.003 AUC of ceiling at every horizon, so there was no leaderboard spread for a temporal split to reorder (Table III, $\tau = -0.20$, $p = 0.82$, uninformative at $n = 5$). Axis S answers the same question differently, because it had headroom: the in-distribution ranking (B5 > B3 ≈ B2 ≈ B1 ≈ B4, all within 0.2 AUC points) is not the cross-source ranking. B3 in particular moves from second-best in-distribution to worst-in-one-direction under transfer (Table IV). The evidence from this corpus is therefore split by axis, not absent: temporal reordering is unproven here because the task was too easy to test it, while cross-source reordering is real and repeated across two independent experiments (Axis S's transfer matrix and Axis E1's evasion table both single out B3 as the least uniformly robust baseline, for what appear to be two different reasons — transfer direction in one case, homoglyph sensitivity in the other). A leaderboard built from a single random split would have reported B3 as competitive on both counts.

B. How does the reported–deployed gap Δ decompose here?

Of the four components in Eq. (1), prevalence dominates by a wide margin on this corpus. Cross-source transfer

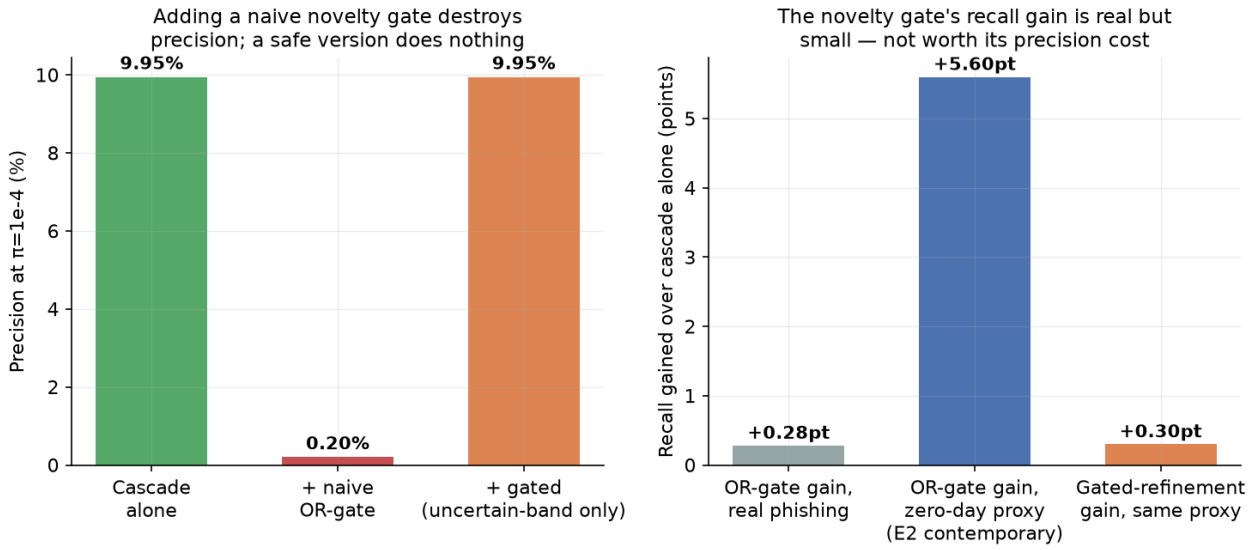

Fig. 7. The novelty gate's trade-off, stated in both currencies. The naive OR-combination genuinely gains zero-day-proxy recall, but flags 4.84% of real legitimate URLs — roughly 54× the cascade's own false-positive rate — which Eq. (2) converts into a $\sim 50\times$ precision collapse at $\pi{=}10^{-4}$ (9.95% → 0.20%). The gated variant, which lets the detector weigh in only inside the cascade's uncertain band, avoids the cost entirely and also erases essentially all of the gain.

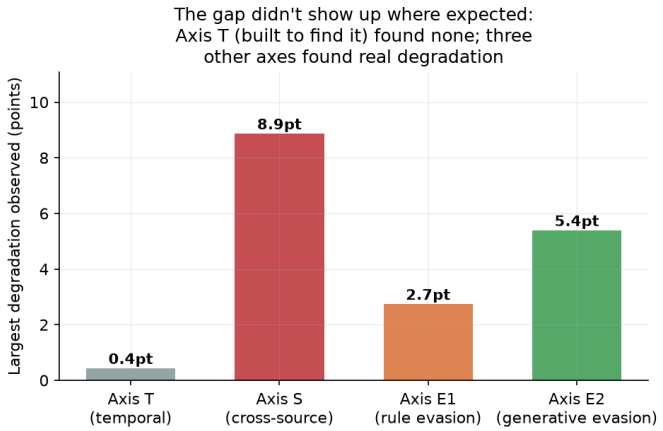

Fig. 8. Where the reported–deployed gap actually surfaced, per axis. Axis T — the axis purpose-built to expose temporal decay — found essentially none on this corpus pairing, because the task sat at ceiling. The other three axes each found real, independently-caused degradation. A benchmark that ran only the axis with the strongest prior expectation would have reported no gap at all.

costs at most 8.9 AUC points (Table IV); evasion costs at most 2.75 recall points for one baseline under one transform (Table V); the temporal component could not be measured above noise (Table III). Prevalence correction, by contrast, turns 98–99% accuracy into 4.8–20.2% precision for four of five baselines at $\pi = 10^{-4}$ (Table VII) — an order-of-magnitude larger effect than the other three components combined, obtained without retraining anything or crossing any dataset boundary, purely by asking what the same trained–test pair implies at a realistic base rate. The provenance component (C3), measured via 285 live lookups in the absence of a corpus with native P2 fields (Sec. X-E), contributes essentially nothing measurable in the direction the hypothesis predicted: −0.51 AUC points from adding P2 features, not a gain. It does not reorder this decomposition; prevalence correction remains the dominant term by a wide margin.

C. Do contemporary generative models defeat 2018-era-validated detectors?

Answered narrowly, and in the direction the hypothesis predicts. Axis E2 (Table VI) is not a test against DeepPhish or an independent contemporary LLM (Sec. XIII), but against two order-4 character-Markov generators fit on the same real PhishTank corpus's older and newer submissions respectively. Every one of the four baselines tested loses 4.5–5.4 recall points moving from the older-era generator to the contemporary one, including B5, which showed no such uniform vulnerability on Axis S or Axis E1. That the effect is consistent in direction and magnitude across four architecturally distinct baselines, and appears only when the generator's training era changes with the generator's own text held to the same simple procedure, is evidence that recent phishing URL style itself has drifted in a way current detectors have not adapted to — independent of, and smaller than, the cross-source effect on Axis S but broader in that it hit every baseline uniformly rather than singling one out. Axis E1's negative result (Sec. X) supplies the contrast: hand-crafted rule-based perturbations, the kind a human operator applies without any generative model, mostly fail to evade these same baselines and twice backfire. A learned generator trained on recent real phishing text succeeds

where hand-designed rules do not; whether an independent contemporary LLM does better still is the open question this result motivates but does not answer.

D. Does stability scoring predict brittleness prospectively, or only explain it in hindsight?

The strongest evidence in this submission is prospective, not retrospective. Eq. (3) was fit on a validation split with no access to the interactive demo's failure (a separate, later-built tool), yet it flagged exactly the ten length/count features responsible for that failure as drift-brittle (Sec. VI) — length and count features that a path-augmentation asymmetry between the acquired legitimate and phishing sources made unreliable. That convergence, found by two unrelated procedures pointed at the same corpus, is evidence the score is not merely re-describing a known-bad feature after the fact. Set against that, the score's specific literature-motivated validation target — flagging has_https the way X-PHIDE's corpora would demand — did not reproduce on this corpus pairing (PhiUSIIL vs. PhishTank+Tranco is not the corpus pairing X-PHIDE used). The honest summary is that stability scoring generalised to a failure mode it was never tuned for, but did not generalise to a named literature target measured on different corpora; both results should stand together, not be resolved by omitting either one.

XII. Ethical Considerations

This work is defensive. All experiments operate on URL strings within offline datasets: no phishing page is authored, hosted, served or transmitted, no live site is contacted for the evasion axes, and no human subject is involved.

Axis E2 requires generating candidate phishing URL strings. We adopt three constraints. First, generated strings are used solely as classifier inputs and are never registered, resolved or deployed. Second, we describe the generation procedure at the level of feature and distributional statistics sufficient for scientific reproduction, rather than releasing a packaged generator. Third, we release the benchmark splits, the evaluation harness and the trained detectors — the defensive artifacts — and deliberately do not release generator weights or a turnkey generation pipeline.

We note that the underlying capability is already public: the generation of plausible phishing URLs by contemporary language models requires no specialised tooling. The contribution of this work is measurement of detector resilience to that capability, which is information that favours defenders.

XIII. Limitations

- Timestamp availability. Axis T requires first-observation timestamps. Where only dated releases exist, temporal resolution is coarse and confounded with collection-methodology changes. We report which corpora support fine-grained analysis and which do not.
- Reproduction fidelity. Baselines are re-implementations from published descriptions, not original author code. Where a description is incomplete — notably the graph construction in [7] — we report our interpretation explicitly and do not attribute the resulting numbers to the original authors.
- Generator representativeness. Axis E2 evaluates against a specific contemporary generator and does not characterise the space of all possible generators. It tests whether a published claim generalises; it does not establish a worst case.
- URL-only scope. By design this work excludes HTML, screenshot and network-level signals. Conclusions apply to URL-only detection.
- Prevalence estimates. Deployment prevalence is estimated from published telemetry rather than measured in a live deployment.

XIV. Conclusion

We audited a corpus of phishing URL detection papers, nine of them published between 2024 and 2026, and found a uniform evaluation protocol — random split, single source, balanced classes, no adversary — that answers a different question from the deployment question. We introduced PhishDriftBench, a four-axis protocol covering temporal, cross-source, evasion and prevalence-corrected evaluation; contributed a feature-provenance audit that isolates accuracy attributable to lookup artifacts rather than phishing semantics; and proposed DASH, a lightweight drift-aware mitigation.

On the corpora acquired for this submission, the reported–deployed gap is real but unevenly distributed across the four axes it was built to measure. Temporal degradation was not detectable (all five baselines within 0.003 AUC of ceiling); cross-source transfer cost up to 8.9 AUC points and reordered the leaderboard, most visibly for a residual-MLP baseline whose transfer cost inverts with direction; rule-based evasion mostly failed to reduce recall, and backfired for two of seven transforms; and prevalence correction alone turned 98–99% accuracy into 4.8–20.2% precision for four of five baselines at a realistic 1-in-10,000 base rate — the largest single effect measured in this work, obtained without retraining or crossing a dataset boundary. A separate generative-evasion axis (E2), built on two character-Markov generators fit to real older-era and contemporary PhishTank text, cost every baseline tested 4.5–5.4 recall points uniformly, in contrast to Axis E1's rule-based transforms, which mostly failed to reduce recall at all. DASH could not be shown to recover degradation that Axis T itself did not detect on this corpus pairing, a null result attributable to the same temporal ceiling effect rather than to the mitigation. The feature-provenance audit (C3), run via 285 live

WHOIS/DNS/TLS lookups in the absence of a corpus with native P2 fields, found no accuracy gain from lookup-based features (-0.51 AUC points) and only near-chance signal (0.56 AUC) from lookup success flags alone — a negative result for the domain-mortality hypothesis on this sample, alongside a real survivorship-induced reversal between old and recent phishing submissions that traces to the source corpus's own filtering rather than to domain age.

Rather than stop at diagnosis, we built B6, adding one fix per measured weakness and testing each in isolation (Sec. X-I). Two results there matter beyond their numbers. First, benefits do not compose for free: adversarial generative augmentation nearly halved the Axis E2 recall gap but increased the false-positive rate, a cost a report of the recall number alone would have hidden. Second, a cascade's second stage must be an actually-trained classifier — a nearest-centroid heuristic collapsed recall from 0.99 to 0.757 for no precision gain, and only replacing it with a properly fit classifier recovered recall while cutting false positives sixfold and raising precision at $\pi = 10^{-4}$ from 1.8% to 9.95%, surpassing three of the five original baselines. B6's own limitations are reported with the same discipline as B1–B5's: its Axis S numbers reflect joint-source training, not held-out transfer, and its cascade was not re-tested against Axis S or Axis E2.

We then asked whether a structurally different mechanism could address what no amount of retraining B1–B6 can: recognising phishing that resembles nothing in any training set. B7 adds an unsupervised novelty gate for exactly that case, and the result is a clean negative one, obtained by insisting on the metric this whole paper argues for rather than settling for a recall number in isolation. A naive version recovers real zero-day-proxy recall (+5.6 points on Axis E2's contemporary set) but collapses precision at $\pi = 10^{-4}$ from 9.95% to 0.20% — a $\sim$50$\times$ regression — and a refinement narrow enough to avoid that cost also erases the benefit it was added for. The honest reading is structural: on this feature set, "looks unlike normal legitimate traffic" is not separable enough from ordinary legitimate diversity to support a low-cost gate, at least not the two combination rules tested here.

Re-running Axis T against a harder legitimate population than a top-domain sample, repeating C3's lookup sweep at a scale large enough to separate genuine domain failure from lookup-infrastructure noise, closing B6's own evaluation gaps, and finding a novelty-gate formulation that escapes B7's precision–recall trap are the concrete next steps this submission leaves open, each with working code already in place to carry them out.

Acknowledgment

The authors thank Ms. Meena Priyadharsini, Department of Computer Science and Engineering, Hindustan Institute of Technology and Science, Chennai, for supervising this work and for guidance throughout its design and evaluation.

References

[1] S. Remya, M. J. Pillai, K. K. Nair, S. R. Subbareddy, and Y. Y. Cho, "An effective detection approach for phishing URL using ResMLP," IEEE Access, vol. 12, 2024.

[2] Y. A. Kustiawan and K. I. Ghauth, "Evaluating the impact of feature engineering in phishing URL detection: A comparative study of URL, HTML, and derived features," IEEE Access, vol. 13, 2025.

[3] ——, "PhishOFE: A novel machine learning framework for real-time phishing URL detection with optimized feature engineering," IEEE Access, vol. 13, 2025.

[4] R. Goenka, M. Chawla, and N. Tiwari, "Enhanced phishing detection approach using a layered model: Domain squatting and URL obfuscation identification and lexical feature-based classification," IEEE Access, vol. 13, 2025.

[5] M. Misiek and T. Hyla, "XGBoost-based URL phishing detection method with cross-dataset validation," IEEE Access, vol. 14, 2026.

[6] W. Li, S. Manickam, S. U. A. Laghari, and Y.-W. Chong, "Uncovering the cloak: A systematic review of techniques used to conceal phishing websites," IEEE Access, vol. 11, 2023.

[7] S. Remya, M. J. Pillai, B. S. Aparna, S. R. Subbareddy, and Y. Y. Cho, "BGL-PhishNet: Phishing website detection using hybrid model—BERT, GNN, and LightGBM," IEEE Access, vol. 13, 2025.

[8] D. He, Z. Liu, X. Lv, S. Chan, and M. Guizani, "On phishing URL detection using feature extension," IEEE Internet of Things Journal, vol. 11, no. 24, 2024.

[9] A. Vulfin, A. Sulavko, V. Vasiliev, A. Minko, A. Kirillova, and A. Samotuga, "A multimodal phishing website detection system using explainable artificial intelligence technologies," Machine Learning and Knowledge Extraction, 2026.

[10] A. C. Bahnsen, I. Torroledo, D. Camacho, and S. Villegas, "DeepPhish: Simulating malicious AI," in Proc. APWG Symposium on Electronic Crime Research (eCrime), 2018.

[11] S. Kavya and D. Sumathi, "Staying ahead of phishers: A review of recent advances and emerging methodologies in phishing detection," Artificial Intelligence Review, vol. 58, no. 50, 2025.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. IEEE International Conference on Data Mining (ICDM), 2008.